Final Defense Report
Task List

May 2026

Bidault Quentin
Le Berre Adrien
Lin Jérôme
Class : A1-C1

1 Context and objectives

1.1 Project Context

In a fast-paced digital world, managing personal and professional obligations requires efficient tools. While command-line tools offer speed, a Graphical User Interface (GUI) provides the visual clarity needed for complex scheduling.

This project aims to develop a robust, cross-platform task management desktop application using Rust. By leveraging Rust's performance and memory safety, we intend to provide a fluid user experience that remains lightweight even when handling large volumes of data and complex features like calendar synchronization.

1.2 Objectives

The primary objective is to build a high-performance task manager with the following specific goals:

- Comprehensive Task Management:

Implement full CRUD operations (Create, Read, Update, Delete) for tasks. Manage rich task details, including due dates, priority levels (Low, Medium, High), and completion status.

- Organization and Categorization:

Allow users to create and delete custom categories to group tasks by project or context. Provide advanced sorting and filtering mechanisms based on date, priority, or category.

- Advanced Scheduling:

Support recurring tasks (daily, weekly, monthly) to automate routine planning. Integrate a Calendar View to visualize deadlines and workload over time.

- Data Security and Portability:

Implement automatic and manual backup systems to prevent data loss. Enable Data Import/Export (e.g., JSON or CSV) to ensure user data remains portable and accessible.

- Modern User Interface:

Develop a responsive and intuitive GUI that ensures a seamless user experience across different operating systems.

2 Functional description

The system provides tools for creating, managing, and organizing tasks, along with mechanisms for storing and handling user data. It defines how users interact with the application and what features are available to support efficient task management.

2.1 Task Management

Users gain access to functions allowing creation, modification and deletion of tasks. A title along with status appears within every task entry. Among choices: pending, in progress, completed one serves as the sole status option. Description or priority might appear alongside required elements. Scaling from optional through critical defines how urgency ranks across levels.

Completion of a repeating task triggers creation of a fresh occurrence, guided by an established repetition pattern. Following such completion, the platform produces another version without manual input. Frequencies available for repetition consist of every day, each week, or once per month. This behavior applies only when rules are already defined within settings.

Categories can be created, changed, or removed by users through the system. Following a category removal, linked tasks remain unless specified otherwise. One at a time, adjustments must be made manually for affected tasks. A task might belong to one category, though it may exist independently if needed. Decisions about unlinked tasks rest solely with the user after deletion.

2.2 Task Organization

A built-in sorting function helps users arrange tasks by different attributes. Depending on preference, entries appear ordered by name, deadline, urgency, or completion state. One option lets people group items alphabetically through their titles. Another path organizes assignments by when they are due.

Priority tags guide arrangement based on importance levels. Status indicators allow viewing progress stages as a sequence.

One way to sort tasks is by their type. A separate screen exists for every group, showing only the items linked to it. Filtering happens automatically when selecting one of these sections. Views are built specifically around each classification. Navigating between them reveals different sets of assignments.

A built-in calendar display will form part of the system. Through this tool, people can see tasks laid out by planned or deadline dates. As a result, managing time becomes more intuitive. The layout supports clearer scheduling decisions across days.

2.3 Data Management

Even if closed, the application retains user information between uses. Information about tasks and groupings remains saved on the device, arranged through organized formats like JSON or CSV files.

Also available is the capability to move data in and out of the system, supporting easier handling across environments. Stored items such as tasks and groupings can exit the platform in file form, useful when shifting platforms or saving copies. Every exchange process maintains accuracy, keeping details unchanged during transfers. Consistency within the database remains intact through each operation.

2.4 User Interface

A clean desktop interface appears by default, built for straightforward interaction. A panel lines the left edge while the primary workspace fills the center. Navigation lives in that vertical strip, where users move to tasks, category sections, or the calendar through clickable entries. Categories appear automatically in the menu.

A user sees tasks laid out plainly, each marked by name, current state, urgency level, and deadline. To add or adjust an item, a button triggers a pop-up where data is filled or changed. The layout remains clean and focused on readability.

A calendar pane displays scheduled tasks across months, with weekly views available if needed. Navigation feels natural and simplicity stays central throughout the arrangement.

2.5 Website

The project's webpage gives a straightforward look at what the app does along with key functions it supports.

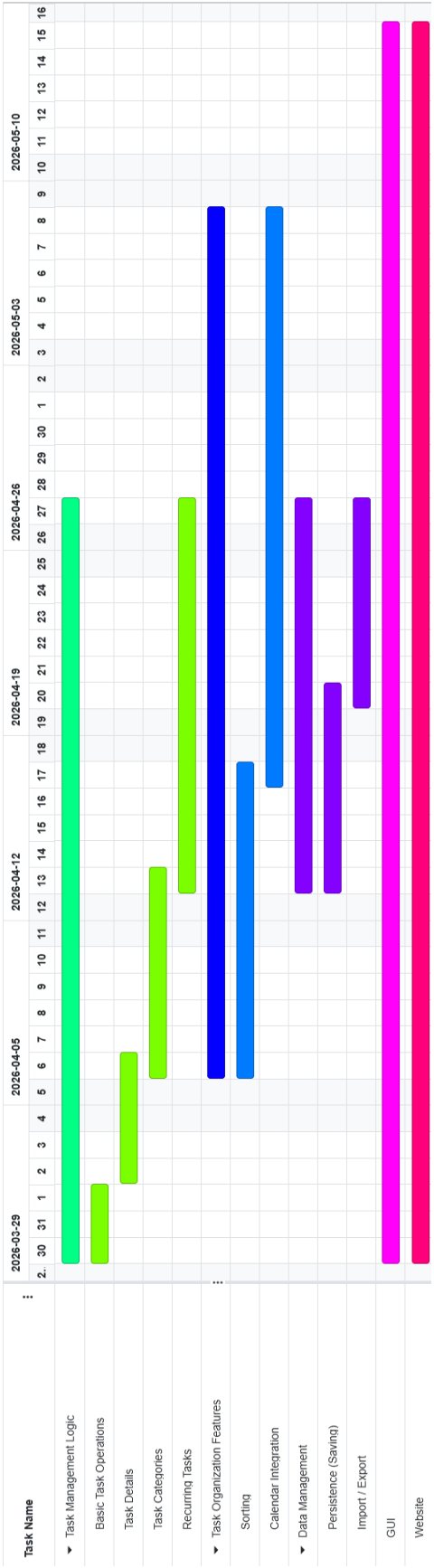
A quick glance reveals how the tool organizes tasks, making daily planning easier for visitors who land there. From there, anyone can grab the latest build. Information on how to use core parts appears early so confusion drops fast once someone begins exploring.

3 Technical constraints

The project will be developed using the Rust programming language, ensuring full compatibility and a consistent experience across both Windows and Linux environments. The development process will rely on Cargo for efficient dependency management and automated build procedures.

4 Project organization

4.1 Task distribution

Task	**Responsible**
Basic Task Operation	Adrien
Task Details	Adrien
Task Categories	Quentin
Reccurring Task	Quentin
Sorting	Quentin
Calendar Integration	Jérôme
Persistence (Saving)	Jérôme
Import/Export	Jérôme
GUI	Jérôme
Website	Adrien

4.2 Gantt Diagram